McMaster University
Engineering

RoCam

High Performance Vision-Guided Rocket Tracker

Capstone Project
Department of
Computing and Software

Motivation

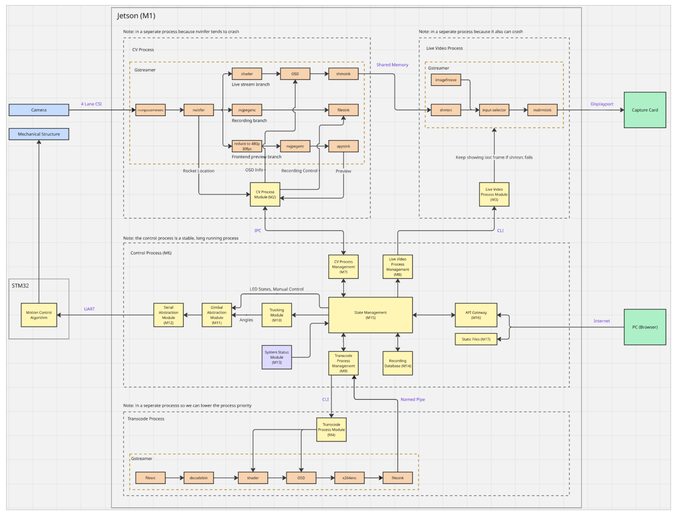
The Problem: Model rockets accelerate to extreme speeds within fractions of a second, making manual camera tracking virtually impossible. Existing commercial tracking systems are cost-prohibitive and lack the precision required for small, fast-moving targets. Consequently, rocketry teams lose critical visual data for mid-flight events like staging and parachute tangling.

The Solution: RoCam bridges this gap with an **autonomous, vision-guided camera tracking system**. By locking onto the rocket in real time, it actively commands a two-axis gimbal to deliver smooth, stabilized **1080p video at 60 fps**.

The Impact: Designed to reliably capture motor ignition, staging, and recovery, RoCam provides actionable flight data. While optimized for small-scale launches (<200 m apogee), the platform is fully **scalable to high-powered rocketry** (3 km+ apogee) for organizations like the McMaster Rocketry Team.

RoCam Launch and Tracking Operations

System Architecture



Real-Time Vision and Control Pipeline

Edge Compute & Hardware. The implemented architecture follows a distributed multi-process edge-compute design on an NVIDIA Jetson. To ensure high reliability, CPU/GPU-heavy perception workloads are isolated from control and operator services via Inter-Process Communication (IPC). This strict fault isolation preserves deterministic command flow and API stability even if the vision pipeline encounters transient errors during launch-dynamic conditions.

Vision Pipeline. Image acquisition utilizes a GStreamer capture graph configured for 1920×1080 at 60 fps, feeding NVIDIA DeepStream for GPU-accelerated preprocessing. Detection relies on a YOLO network optimized with TensorRT, yielding low-latency normalized bounding boxes. A decoupled Live Video process parallelizes OSD telemetry overlay and HDMI output to guarantee continuous situational awareness without blocking inference.

Kinematic Control. The tracking module calculates image-plane target error (targeting frame center $cx = 0.5, cy = 0.5$) to generate proportional 2-axis gimbal actuation commands. These commands pass through a validated, state-gated pathway—enforcing rate limits and safety bounds—before transmission to an STM32 microcontroller via UART, ensuring deterministic, closed-loop responsiveness throughout boost and coast regimes.

Backend & State Management. System orchestration is governed by a centralized State Machine (Disarmed, Armed, Tracking, Recording) that prevents invalid mode combinations and enforces hardware safety. A Flask-based API Gateway exposes a REST interface for offline-first LAN operation, managing mode transitions, asynchronous recording/transcoding workflows, and streaming SSE telemetry for live state and control-stream observability.

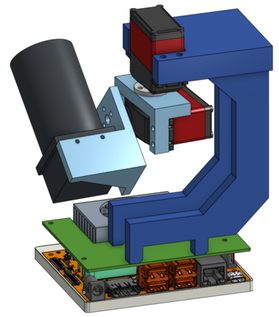
Tools & Technologies

NVIDIA · AUTODESK · GitHub · Firebase · React · TypeScript · Python · Rust

Software & Hardware Features

Web-Based Operator Interface

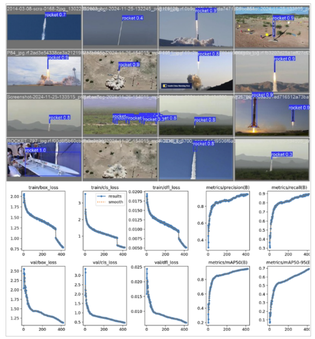
Real-Time Gimbal Control: RoCam provides a low-latency control path between the operator interface and an STM32-based pan–tilt gimbal controller, allowing software commands to be translated into immediate physical camera motion during launch preparation and flight operation. **Manual Positioning:** When the system is in **idle** mode, the operator can use the web interface to manually pan and tilt the camera in real time, making it possible to align the view with the launch rail, expected ascent corridor, or surrounding airspace before ignition. **Remote Operation:** All core control actions are available through a browser-based interface over a local network, so the system can be operated from a safe stand-off distance without requiring physical access to the tracker or any public internet connection. **System-State Awareness:** The interface presents clear visual feedback on the current state of the system, including **idle**, **armed**, and **tracking**, allowing the operator to quickly verify readiness and respond appropriately during time-critical launch procedures. **Safe Override:** At any point, the operator can immediately stop automated motion and command the system to return to a safe **idle** state, which is essential for fault recovery, unexpected target behavior, or general operational safety.



Two-Axis Gimbal Prototype

Autonomous Vision Tracking: Once the system is explicitly placed in the **armed** state, RoCam continuously processes the live camera feed to detect the rocket as a small, fast-moving target under realistic outdoor launch conditions. **Automatic State Transition:** When a valid rocket target is detected, the system automatically transitions from **armed** to **tracking** without requiring further operator input, reducing reaction delay during the most critical phase of flight. **Closed-Loop Target Following:** Detection results from the vision pipeline are converted into continuous pan and tilt corrections so that the gimbal actively follows the rocket and keeps it near the center of the frame throughout ascent and, where possible, descent. **Robustness in Challenging Conditions:** The tracking pipeline is designed to remain effective despite common disturbances such as smoke plume, glare, motion blur, or rapid angular movement, all of which are expected in real launch environments. **Short-Term Re-Acquisition:** If the rocket is briefly lost from view, the system attempts short-term re-acquisition before declaring the target lost, improving overall tracking continuity and increasing the likelihood of preserving useful flight footage without manual intervention.

Computer Vision & Machine Learning



YOLO26s-P2 Training Performance

- **Architecture Upgrade (YOLO26s-P2):** Added a **stride-4 detection head** to quadruple the spatial resolution of the finest feature map, ensuring sensitivity to micro-targets as small as 10×10 pixels.
- **Data Engineering & Hard Negative Mining:** Engineered an automated pipeline to inject **4,000 pure background COCO images** (∼25% of the dataset) with empty labels, forcing a harder decision boundary to suppress false positives.
- **Custom Degradation Pipeline:** Integrated a DDP-compatible Albumentations pipeline to simulate real-world flight conditions, including motion blur, sensor noise, dynamic range shifts, and compression artifacts.
- **Curriculum Augmentation Strategy:** Implemented a progressive reduction strategy across training stages—shifting from heavy augmentation (exploration) to minimal perturbation (exploitation).
- **Failure Analysis & Debugging:** Diagnosed a critical 'Mosaic Starvation' failure where early stopping preempted the `close_mosaic` phase, trapping the model on $\frac{1}{4}$-scale downscaled images and crashing mAP_{50-95} to 0.614.
- **Close-Mosaic Fine-Tuning:** Engineered a dedicated recovery stage (mosaic=0.0) to expose the model to full-resolution 960×544 frames, rescuing the pipeline and triggering a massive +17.3% metric jump.
- **HPC Distributed Training:** Orchestrated the complete four-stage training pipeline on a **4×NVIDIA H100 PCIe (81 GB)** cluster using Distributed Data Parallel (DDP) and mixed precision.

Final Deployment Metrics: $\mathbf{mAP}_{50} = 0.958$; $\mathbf{mAP}_{50-95} = 0.750$

Our Team

Development Team

RoCam

Zifan Si · Jianqing Liu
Mike Chen · Xiaotian Lou

Launch Team

McMaster Rocketry Team

Advisory Team

Dr. Shahin Sirouspour · Dr. Spencer Smith
Tanya Djavaherpour